ENTREGA 1

INTEGRACIÓN CONTINUA

Autor:

Francisco Javier Betancur Moreno

Ingeniería de Software, Politécnico Grancolombiano

Tutor:

John Olarte Ramos

Noviembre 2026

Contenido

1. Introducción y Objetivos de la Entrega 3

2. Arquitectura del Proyecto y Configuración de Contenedores 4

3. Evidencias y Verificación Técnica 4

4. Conclusiones 7

5. Referencias Bibliográficas 7

1. Introducción y Objetivos de la Entrega

La integración continua (CI) representa un pilar fundamental en el desarrollo moderno de software, permitiendo mitigar errores de integración de forma temprana mediante la automatización de compilaciones, pruebas y despliegues. El presente informe documenta el desarrollo y la implementación técnica de la Entrega 1 (Escenario 3) del módulo de Énfasis Profesional I, aplicada sobre el repositorio base de la plataforma web de noticias (newsweb-platform).

El objetivo principal de esta fase inicial es el diseño y despliegue local de una arquitectura basada en contenedores Docker, garantizando la separación de ambientes, el uso de versiones fijas en las imágenes, la gestión segura de credenciales y variables de entorno, y la comunicación efectiva entre contenedores dentro de una red privada virtual.

2. Arquitectura del Proyecto y Configuración de Contenedores

Para dar cumplimiento a los requerimientos de la entrega, se estructuró una arquitectura de dos servicios principales orquestados mediante Docker Compose:

• **Contenedor 1 (Frontend / Aplicación Web):** Construido a partir de un Dockerfile propio de múltiples etapas (multi-stage build), utilizando Node.js para la compilación del código fuente en Angular y Nginx (versión fija 1.25.3-alpine) para servir los archivos estáticos resultantes.

• **Contenedor 2 (Servicio Base de Datos / API):** Implementado con PostgreSQL (versión fija 15.5-alpine), configurado con variables de entorno segregadas y un volumen persistente para garantizar la integridad y almacenamiento seguro de los datos.

Diagrama Conceptual de Arquitectura de Red



3. Evidencias y Verificación Técnica

A continuación se presentan las evidencias documentadas mediante capturas de pantalla de la ejecución y validación del entorno en el sistema operativo:

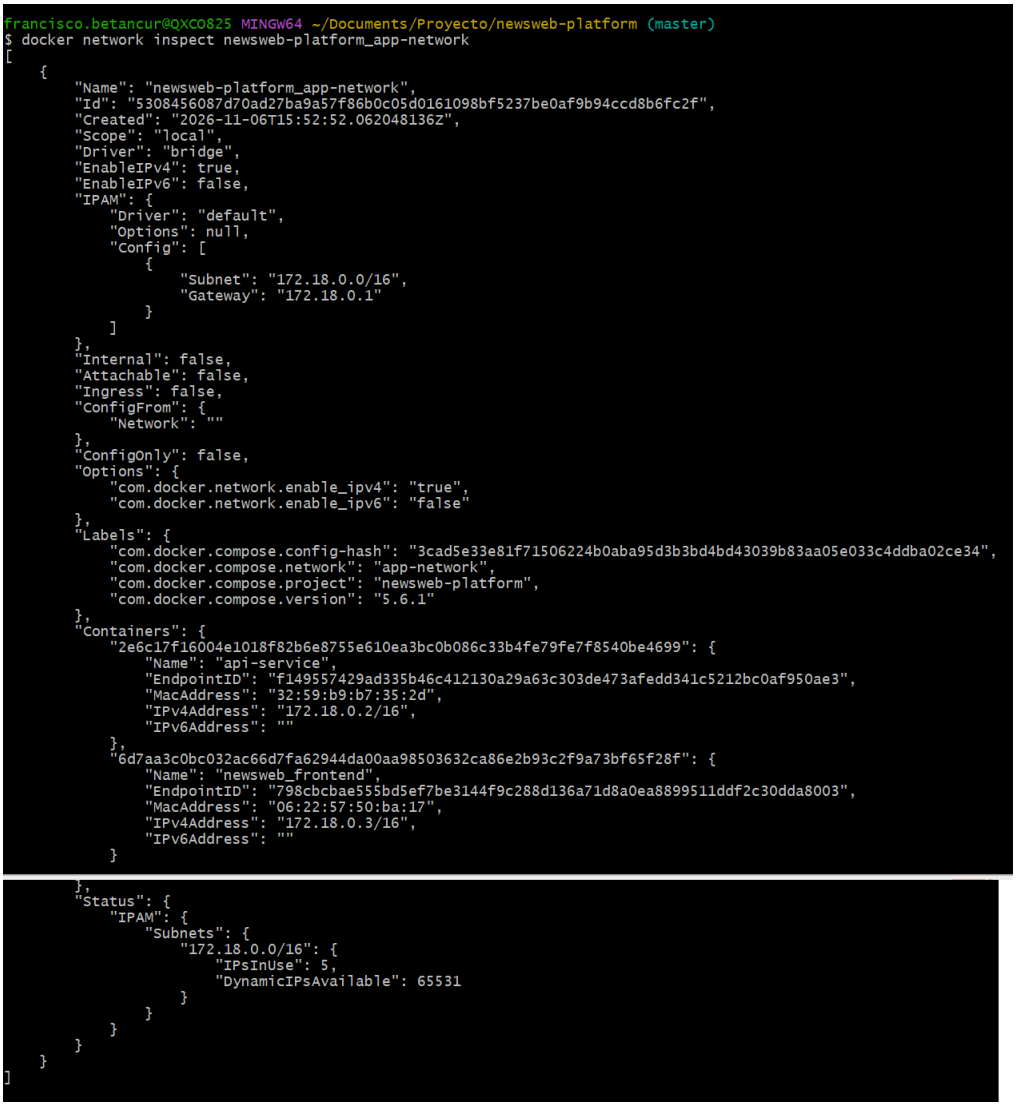
Figura 1: Validación del estado de los contenedores (docker ps)

Se ejecuta el comando docker ps para comprobar que tanto el contenedor frontend (newsweb_frontend) como el de base de datos (api-service) se encuentran activos y ejecutándose correctamente en estado 'Up'.

```
francisco.betancur@QXCO825 MINGW64 ~/Documents/Proyecto/newsweb-platform (master
)
$ docker ps
CONTAINER ID   IMAGE                          COMMAND                  CREATED
 STATUS        PORTS                                        NAMES
6d7aa3c0bc03   newsweb-platform-web-app       "/docker-entrypoint.…"   3 hours ago
 Up 3 hours    0.0.0.0:4200->80/tcp, [::]:4200->80/tcp      newsweb_frontend
2e6c17f16004   postgres:15.5-alpine           "docker-entrypoint.s…"   3 hours ago
 Up 3 hours    5432/tcp                                     api-service
```

Figura 2: Inspección de la red personalizada de Docker

Mediante el comando docker network inspect newsweb-platform_app-network, se verifica la creación y configuración de la red tipo bridge que aísla y conecta de manera segura ambos servicios.



```
francisco.betancur@QXCO825 MINGW64 ~/Documents/Proyecto/newsweb-platform (master)
$ docker network inspect newsweb-platform_app-network
[
    {
        "Name": "newsweb-platform_app-network",
        "Id": "5308456087d70ad27ba9a57f86b0c05d0161098bf5237be0af9b94ccd8b6fc2f",
        "Created": "2026-11-06T15:52:52.062048136Z",
        "Scope": "local",
        "Driver": "bridge",
        "EnableIPv4": true,
        "EnableIPv6": false,
        "IPAM": {
            "Driver": "default",
            "Options": null,
            "Config": [
                {
                    "Subnet": "172.18.0.0/16",
                    "Gateway": "172.18.0.1"
                }
            ]
        },
        "Internal": false,
        "Attachable": false,
        "Ingress": false,
        "ConfigFrom": {
            "Network": ""
        },
        "ConfigOnly": false,
        "Options": {
            "com.docker.network.enable_ipv4": "true",
            "com.docker.network.enable_ipv6": "false"
        },
        "Labels": {
            "com.docker.compose.config-hash": "3cad5e33e81f71506224b0aba95d3b3bd4bd43039b83aa05e033c4ddba02ce34",
            "com.docker.compose.network": "app-network",
            "com.docker.compose.project": "newsweb-platform",
            "com.docker.compose.version": "5.6.1"
        },
        "Containers": {
            "2e6c17f16004e1018f82b6e8755e610ea3bc0b086c33b4fe79fe7f8540be4699": {
                "Name": "api-service",
                "EndpointID": "f149557429ad335b46c412130a29a63c303de473afedd341c5212bc0af950ae3",
                "MacAddress": "32:59:b9:b7:35:2d",
                "IPv4Address": "172.18.0.2/16",
                "IPv6Address": ""
            },
            "6d7aa3c0bc032ac66d7fa62944da00aa98503632ca86e2b93c2f9a73bf65f28f": {
                "Name": "newsweb_frontend",
                "EndpointID": "798cbcbae555bd5ef7be3144f9c288d136a71d8a0ea8899511ddf2c30dda8003",
                "MacAddress": "06:22:57:50:ba:17",
                "IPv4Address": "172.18.0.3/16",
                "IPv6Address": ""
            }
        },
        "Status": {
            "IPAM": {
                "Subnets": {
                    "172.18.0.0/16": {
                        "IPsInUse": 5,
                        "DynamicIPsAvailable": 65531
                    }
                }
            }
        }
    }
]
```

Figura 3: Ingreso interactivo al contenedor Frontend

Se establece una sesión interactiva (sh) dentro del contenedor newsweb_frontend para validar el entorno interno y realizar pruebas directas de conectividad de red.

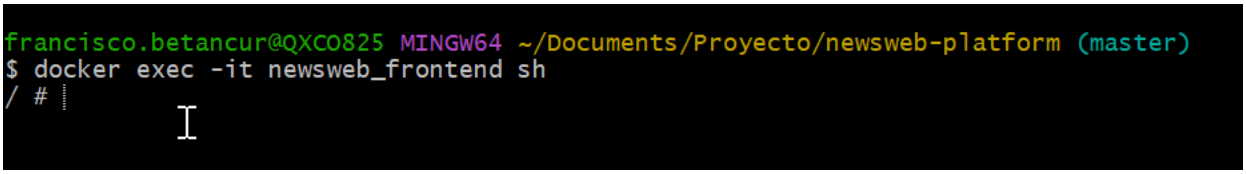


Figura 4: Prueba de comunicación por Nombre de Servicio (Punto Central)

Se ejecuta la prueba de red nc -zv api-service 5432 desde el contenedor frontend hacia el servicio de base de datos. La respuesta 'open' confirma el éxito de la comunicación utilizando estrictamente el nombre del servicio dentro de la red Docker.

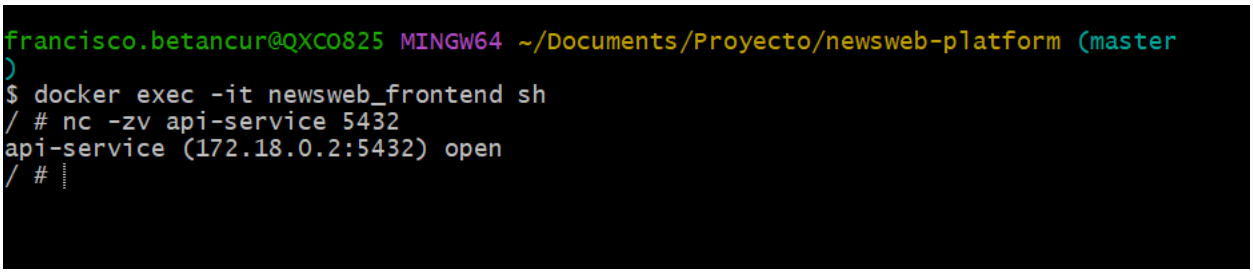


Figura 5: Prueba de interacción y transacción de ida y vuelta en PostgreSQL (Punto Central)

Se establece conexión interactiva mediante psql hacia el servicio de base de datos (api-service), ejecutando la creación de una tabla de prueba, la inserción de un registro y su posterior consulta (SELECT). Esto evidencia de manera concluyente la operación funcional y el almacenamiento persistente de ida y vuelta en la arquitectura.

```
PS C:\Users\francisco.betancur\Documents\Proyecto\newsweb-platform> docker exec -it api-service psql -U postgres
Type "help" for help.

postgres=# CREATE TABLE IF NOT EXISTS prueba_conexion (id SERIAL PRIMARY KEY, mensaje VARCHAR(100));
CREATE TABLE
postgres=# INSERT INTO prueba_conexion (mensaje) VALUES ('Prueba de comunicacion exitosa frontend-backend');
INSERT 0 1
postgres=# SELECT * FROM prueba_conexion;
 id |                    mensaje
----+-----------------------------------------------
  1 | Prueba de comunicacion exitosa frontend-backend
(1 row)

postgres=#
```

URL del repositorio: https://github.com/FRANBETM/newsweb-platform.git

4. Conclusiones

1. La implementación de Docker y Docker Compose permitió estandarizar el entorno de desarrollo y despliegue, eliminando la variabilidad entre equipos de trabajo y asegurando un comportamiento idéntico en las fases de pruebas y producción.

2. El uso de nombres de servicio dentro de una red privada tipo bridge demostró ser un mecanismo robusto y eficiente para la comunicación interna entre contenedores, evitando dependencias frágiles de direcciones IP fijas.

3. La adopción de buenas prácticas de seguridad, como el uso de versiones fijas en las imágenes base, el empleo de .dockerignore y la segregación de credenciales mediante variables de entorno (.env), garantiza la calidad y mantenibilidad del proyecto.

5. Referencias Bibliográficas

Oliveros Acosta, E. E. (2017). Lectura fundamental 1: Fundamentos de la Integración Continua. Politécnico Grancolombiano.

Oliveros Acosta, E. E. (2017). Lectura fundamental 2: Manejo de repositorios. Politécnico Grancolombiano.

Oliveros Acosta, E. E. (2017). Lectura fundamental 3: Máquinas virtuales y Dockers para construcción de ambientes. Politécnico Grancolombiano.

Oliveros Acosta, E. E. (2017). Lectura fundamental 4: Uso de contenedores en DEVOPS. Politécnico Grancolombiano.

Fowler, M. (2006). Continuous Integration. MartinFowler.com.